Introduction

Interviewers love problems like LeetCode 1385 because they look trivial on the surface and quietly test whether you reach for the first idea that works or the idea that scales. The brute force here is one that almost every candidate writes in the first thirty seconds — nested loops comparing every element of one array against every element of another. It's correct. It also happens to be exactly the kind of answer that makes an interviewer ask, "can you do better?"

That follow-up question is the whole point of the exercise. Can you recognize that sorting one array changes the nature of the lookup from "scan everything" to "jump straight to the neighborhood"? Can you translate that insight into working code with `bisect`? This problem is a compact, low-risk way to practice exactly that kind of thinking, and the pattern — sort once, binary search per lookup — shows up constantly in real systems: deduplication, nearest-neighbor matching, rate limiting windows, and more.

Problem Overview

You're given two integer arrays, `arr1` and `arr2`, and a non-negative integer `d`. For every value in `arr1`, you need to decide whether it's "safe" — meaning there is no value in `arr2` within a distance of `d` from it. Distance here just means absolute difference.

If a value in `arr1` has at least one value in `arr2` whose absolute difference with it is less than or equal to `d`, that value is disqualified. Otherwise, it counts toward your answer. The function returns the total count of values in `arr1` that survive this check.

Think of it as drones and no-fly zones: `arr1` holds drone positions, `arr2` holds beacon positions, and `d` is the radius around each beacon that a drone must avoid. A drone only counts as "safe" if it clears every single beacon by more than `d`.

Example

```
arr1 = [4, 5, 8]
arr2 = [10, 9, 1, 8]
d = 2
```

- **4**: closest value in `arr2` is... checking all of them, nothing is within 2. Safe.

- **5**: same story — nothing in `arr2` lands within distance 2. Safe.
- **8**: `arr2` contains `8` itself, a distance of 0, which is ≤ 2. Not safe.

Result: **2** safe values (`4` and `5`).

Intuition

The instinct every engineer starts with is "check each value in `arr1` against every value in `arr2`." That's correct, and it's also the first thing to question once you notice a pattern: for any given number, the *only* candidates that could possibly be within distance `d` are the numbers closest to it. You don't need to check the whole array — you need to check its neighbors.

That reframes the problem. If `arr2` is sorted, then for any value `x` , the elements most likely to be close to `x` are sitting right next to where `x` would be inserted into that sorted array. Everything else in `arr2` is farther away by definition, because the array is ordered.

This is the moment binary search becomes the obvious tool: it answers "where would `x` fit in this sorted array?" in logarithmic time. Once you have that insertion point, you only need to examine the element at that index and the one immediately before it — the two closest possible matches. If both are farther than `d` , nothing else in the array can be closer, and `x` is safe.

Brute Force Solution

Idea: For every element in `arr1` , loop through every element in `arr2` and compute the absolute difference. If any difference is ≤ `d` , the element is unsafe.

Algorithm: 1. Initialize `count = 0` . 2. For each `x` in `arr1` , check every `y` in `arr2` . 3. If `abs(x - y) <= d` for any `y` , skip `x` . 4. Otherwise, increment `count` .

Advantages: Dead simple, no sorting, easy to verify by hand.

Disadvantages: Compares every pair, which is wasteful once arrays grow.

Complexity: Time O(n·m), Space O(1).

Optimal Solution

Sort `arr2` once — that costs `O(m log m)` up front. Now, for each value `x` in `arr1` , use `bisect_left` to find the index where `x` would be inserted into the sorted `arr2` . That index tells

you the position of the smallest element ≥ `x`. Check that element (if it exists) and the element just before it (if it exists) — those are the only two candidates that could be within `d` of `x`.

Step	What happens
Sort `arr2`	One-time cost, enables binary search
`bisect_left(arr2, x)`	Finds insertion index `i` for `x`
Check `arr2[i]`	The smallest value ≥ `x`
Check `arr2[i-1]`	The largest value < `x`
Both farther than `d`?	`x` is safe

Because `arr2` is sorted, no element outside these two neighbors can be closer to `x`. That's what collapses the inner loop from `m` comparisons to a single `log m` lookup.

Python Code

```
from bisect import bisect_left
from typing import List


def find_the_distance_value(arr1: List[int], arr2: List[int], d: int) -> int:
    sorted_arr2 = sorted(arr2)
    m = len(sorted_arr2)
    count = 0

    for x in arr1:
        i = bisect_left(sorted_arr2, x)
        safe = True

        if i < m and abs(sorted_arr2[i] - x) <= d:
            safe = False
        if i > 0 and abs(sorted_arr2[i - 1] - x) <= d:
            safe = False

        if safe:
            count += 1

    return count
```

Code Walkthrough

- `sorted_arr2 = sorted(arr2)` — sort once, reused for every lookup.
- `bisect_left(sorted_arr2, x)` — returns the index where `x` would be inserted to keep the array sorted; this is the position of the first element ≥ `x`.

- The `i < m` check guards against `x` being larger than every element in `arr2` (insertion point would be past the end).
- The `i > 0` check guards against `x` being smaller than every element (nothing before index 0).
- Each `abs(...) <= d` check disqualifies `x` if that specific neighbor is too close.
- `count` only increments if neither neighbor triggered a disqualification.

Dry Run

`arr1 = [4, 5, 8]`, `arr2 = [10, 9, 1, 8]`, `d = 2` `sorted_arr2 = [1, 8, 9, 10]`

- `x = 4`: `bisect_left` → index 1 (would sit before 8). Check `arr2[1]=8`, diff=4 > 2, safe so far. Check `arr2[0]=1`, diff=3 > 2, still safe. Count → 1.
- `x = 5`: `bisect_left` → index 1. Check `8`, diff=3 > 2. Check `1`, diff=4 > 2. Count → 2.
- `x = 8`: `bisect_left` → index 1 (matches existing 8). Check `arr2[1]=8`, diff=0 ≤ 2. Unsafe. Count stays 2.

Final result: **2**, matching the expected output.

Complexity Analysis

- **Time:** Sorting `arr2` costs `O(m log m)`. Each of the `n` elements in `arr1` triggers one binary search, `O(log m)`. Total: `O((n + m) log m)`.
- **Space:** `O(m)` for the sorted copy of `arr2`.
- **Why it's correct:** Once `arr2` is sorted, the two elements adjacent to the insertion point are provably the closest possible candidates — any other element is farther away by the ordering property of a sorted array. Checking both covers every case where a closer match could exist on either side of `x`.

Alternative Solutions

You could skip sorting and instead use a two-pointer merge-style scan if both arrays were sorted from the start, but since only one array needs sorting here, the binary search approach is simpler and doesn't require touching `arr1` at all. A brute-force-with-early-exit (breaking out of the inner loop the moment a match is found) helps a little in practice but doesn't change the worst-case complexity, so it's not a real alternative for larger inputs.

Edge Cases

- **d = 0:** An element only counts if no value in `arr2` equals it exactly.
- **Empty `arr2`:** Every element in `arr1` is automatically safe (no beacons to avoid).
- **Duplicate values in `arr2`:** Sorting handles duplicates fine; the binary search still finds a valid neighbor.
- **All elements in `arr1` far from everything:** The full length of `arr1` is returned.
- **Very large or very small values at the boundaries (0 to 10^5 per constraints):** No special handling needed since Python integers don't overflow.

Common Mistakes

1. Checking only `bisect_left`'s index and forgetting to check the element *before* it.
2. Forgetting bounds checks (`i < m`, `i > 0`), causing an `IndexError`.
3. Sorting `arr1` instead of `arr2` — you must sort the array you're searching *into*, not the array you're iterating over.
4. Using `<` instead of `<=` when comparing distance to `d` — the problem says "within d," which is inclusive.
5. Re-sorting `arr2` inside the loop instead of once outside it, silently turning the solution back into something slower than intended.

Interview Questions

- How would this change if `d` could be negative?
- What if you needed the actual safe values, not just the count?
- Could you solve this without sorting, using a different data structure (e.g., a balanced BST or a sorted set)?
- How would you handle streaming data where `arr2` changes over time?

Similar Problems

- **LeetCode 35 (Search Insert Position):** Same core use of `bisect_left`.
- **LeetCode 300 (Longest Increasing Subsequence):** Also leans on binary search over a maintained sorted structure.
- **LeetCode 658 (Find K Closest Elements):** Same "closest neighbors in a sorted array" idea, extended to k elements.

Key Takeaways

Sorting one side of a comparison problem and reaching for binary search is a pattern worth internalizing — it turns "check everything" into "check two neighbors." The win isn't shrinking the loop by a constant factor; it's changing the shape of the lookup from linear to logarithmic.

Watch the Video

For the full walkthrough with the drone-and-beacon visualization, timed benchmarks, and two quiz checkpoints, watch the video here: {LINK}

About the Series

This breakdown is part of *Fun with Learning Technology*, a series dedicated to making coding interview concepts and everyday software engineering ideas intuitive — not just memorized. Each episode pairs a real-world analogy with a full code walkthrough so the "why" sticks as well as the "how."

Call To Action

If this helped the problem click, give the video a like on YouTube and subscribe for more interview-ready breakdowns. For deeper write-ups like this one delivered straight to your inbox, subscribe to the Substack. Drop a comment with questions or problems you'd like covered next, and share this with anyone prepping for interviews.

Quick Review

Pattern name/trigger: comparing every element in A against all of B?

Brute-force pairwise comparison — trigger: small arrays, need per-element count against threshold distance.

Time complexity of brute force vs sort+binary-search approach?

Brute force O(n*m); sort B then binary search per A element gives O(m log m + n log m).

Space complexity of the sorted binary-search solution?

O(m) if sorting a copy, O(log m) extra beyond input if sorting in place.

Trickiest edge case when checking distance condition?

Must use strict '<=' on absolute difference; off-by-one on boundary values (d itself) silently breaks the count.

Linear scan vs binary search here: when does sorting pay off?

Sorting wins when B is large and reused per A element; for tiny B, linear scan per element is simpler and fast enough.

Interview follow-up: how to extend if B changes dynamically between queries?

Maintain B in a balanced structure (sorted list/BST) supporting insert + nearest-neighbor lookup, e.g. a sorted container with bisect-equivalent.
